PYTHON CHALLENGE

List Operations & List Copying

Notes recreated from pages 130–138 of the provided Python Full Notes PDF

Topics included

List slicing • List comprehensions • Nested lists • Sorting and reversing • Copying lists • Shallow copy vs deep copy • Important list operations

GitHub-ready study notes

9.1.5 Slicing

Syntax

```
list_name[start:end:step]
```

Explanation

• Slicing is used to get a part of a list.

• The start index is included.

• The end index is excluded.

• Step controls the gap between selected items.

Example 1

```
numbers = [10, 20, 30, 40, 50, 60]
print(numbers[1:4])
print(numbers[:3])
print(numbers[3:])
print(numbers[::2])
print(numbers[::-1])
```

Output

```
[20, 30, 40]
[10, 20, 30]
[40, 50, 60]
[10, 30, 50]
[60, 50, 40, 30, 20, 10]
```

Slicing Table

Code	Meaning	Result
numbers[1:4]	Index 1 to before 4	[20, 30, 40]
numbers[:3]	Start to before index 3	[10, 20, 30]
numbers[3:]	Index 3 to end	[40, 50, 60]
numbers[::2]	Every second item	[10, 30, 50]
numbers[::-1]	Reverse list	[60, 50, 40, 30, 20, 10]

9.1.6 List Comprehensions

Syntax

```
new_list = [expression for item in sequence]

# With condition:
new_list = [expression for item in sequence if condition]
```

List comprehension is a short way to create a new list. It is commonly used when each item needs to be processed. It can also filter values using `if`. It makes code shorter and cleaner.

Example 1

```
numbers = [1, 2, 3, 4, 5]
squares = [number * number for number in numbers]
print(squares)
```

```
[1, 4, 9, 16, 25]
```

Example 2: With Condition

```
numbers = [1, 2, 3, 4, 5, 6]
```

```
even_numbers = [number for number in numbers if number % 2 == 0]
print(even_numbers)
```

```
[2, 4, 6]
```

9.1.7 Nested Lists

Syntax

```
list_name = [[item1, item2], [item3, item4]]
```

A nested list means a list inside another list. It is useful for matrix-like data, rows and columns, or grouped values. To access nested list values, use multiple indexes.

Example 1

```
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]
print(matrix[0])
print(matrix[1][2])
```

```
[1, 2, 3]
6
```

Updating Nested List Value

```
matrix = [
    [1, 2, 3],
    [4, 5, 6]
]
matrix[1][0] = 40
print(matrix)
```

```
[[1, 2, 3], [40, 5, 6]]
```

9.1.8 Sorting and Reversing

Sorting means arranging values in order. Reversing means changing the order from last to first.

Operation	Code	Changes Original List?	Result
Sort ascending	numbers.sort()	Yes	Small to large
Sort descending	numbers.sort(reverse=True)	Yes	Large to small
Sorted copy	sorted(numbers)	No	Returns new sorted list
Reverse original	numbers.reverse()	Yes	Reverses same list
Reverse copy	numbers[::-1]	No	Returns reversed copy

Example 1

```
numbers = [40, 10, 30, 20]
numbers.sort()
print(numbers)

numbers.sort(reverse=True)
print(numbers)

numbers.reverse()
print(numbers)
```

```
[10, 20, 30, 40]
[40, 30, 20, 10]
```

```
[10, 20, 30, 40]
```

sort() vs sorted()

Point	sort()	sorted()
Type	List method	Built-in function
Original list	Changes original list	Does not change original list
Return value	Returns None	Returns new sorted list
Usage	numbers.sort()	sorted(numbers)

Example 2

```
numbers = [30, 10, 20]
new_numbers = sorted(numbers)
print(numbers)
print(new_numbers)
```

```
[30, 10, 20]
[10, 20, 30]
```

9.1.9 Copying Lists

Syntax

```
new_list = old_list.copy()
```

Other ways:

```
new_list = old_list[:]
new_list = list(old_list)
```

Copying means creating another list with the same values. Direct assignment does not create a real copy. Direct assignment makes two variables point to the same list. To create a separate list, use `copy()`, slicing, or `list()`.

Direct Assignment

```
list1 = [10, 20, 30]
list2 = list1

list2.append(40)
print(list1)
print(list2)
```

```
[10, 20, 30, 40]
[10, 20, 30, 40]
```

Explanation

• list2 = list1 does not create a new list.

• Both variables refer to the same list.

• So changing list2 also affects list1.

Real Copy Example

```
list1 = [10, 20, 30]
list2 = list1.copy()
list2.append(40)
print(list1)
print(list2)
```

```
[10, 20, 30]
[10, 20, 30, 40]
```

Copying Methods Table

Method	Code	Meaning
copy()	new = old.copy()	Creates shallow copy
Slicing	new = old[:]	Creates shallow copy
list()	new = list(old)	Creates shallow copy
Direct assignment	new = old	Not a real copy

9.1.10 Shallow Copy vs Deep Copy

This concept matters when a list contains another list. There are two types of copy: shallow copy and deep copy.

Shallow Copy

A shallow copy creates a new outer list, but nested lists inside are still shared.

Example 1: Shallow Copy

```
list1 = [[10, 20], [30, 40]]
list2 = list1.copy()
list2[0][0] = 99
print(list1)
print(list2)
```

```
[[99, 20], [30, 40]]
[[99, 20], [30, 40]]
```

list1.copy() creates a new outer list, but the inner lists are still shared. So changing an inner list affects both lists.

Deep Copy

A deep copy creates a completely separate copy of both the outer list and inner lists.

Example 2: Deep Copy

```
import copy
list1 = [[10, 20], [30, 40]]
list2 = copy.deepcopy(list1)
list2[0][0] = 99
print(list1)
print(list2)
```

```
[[10, 20], [30, 40]]
[[99, 20], [30, 40]]
```

copy.deepcopy() creates a fully independent copy. Inner lists are also copied separately. Changing list2 does not affect list1.

Shallow Copy vs Deep Copy Table

Point	Shallow Copy	Deep Copy
Outer list	New copy	New copy
Inner nested lists	Shared	New copy
Affects original nested data?	Yes	No
Common method	copy()	copy.deepcopy()
Useful for	Simple lists	Nested lists

Important List Operations Table

Operation	Code	Meaning
Create list	items = [10, 20, 30]	Creates list
Access item	items[0]	Gets first item
Update item	items[1] = 25	Changes second item
Add item	items.append(40)	Adds at end
Insert item	items.insert(1, 15)	Adds at index
Remove item	items.remove(20)	Removes value
Remove by index	items.pop(0)	Removes by index
Slice list	items[1:3]	Gets part of list
Sort list	items.sort()	Sorts original list
Reverse list	items.reverse()	Reverses original list
Copy list	items.copy()	Creates shallow copy
Length	len(items)	Counts items

List Method Return Behavior

Method	Changes Original List?	Returns
append()	Yes	None
extend()	Yes	None
insert()	Yes	None
remove()	Yes	None
pop()	Yes	Removed item
clear()	Yes	None
sort()	Yes	None
reverse()	Yes	None
copy()	No	New list